

[Paper Review] Segment Anything

25기 분석

서영우

0. Abstract

- 이미지 분할 목적, 즉 프롬프트 기반 분할 task를 위해 설계된 모델.

1. Introduction & Segmentation?

- 기존의 분할 과제 유형

1) Semantic Segmentation

각 픽셀을 사전에 정의된 클래스 중 하나로 분류하고자 했으나, 개별 객체를 구별하지 못함.

2) Instance Segmentation

픽셀 분류와 객체 탐지(localization)를 통합하여, 같은 클래스 내에서도 개별 객체 인스턴스를 구분 (Object detection + segmentation 결합 형태)

3) Panoptic Segmentation (Holistic Segmentation)

Semantic(배경, 질감 같은거)과 Instance(사물, 사람, 자동차)를 통합하여 모든 픽셀에 클래스와 인스턴스 ID를 부여. 이미지의 모든 픽셀을 빠짐없이 labeling 진행

그러나 배경과 객체 간의 구분이 아쉬움을 어느정도 해결했으나, 두 개의 파이프라인 (semantic, instance)을 따로 학습하는 구조여서 효율성이 부족한 단점 존재.

2. Promptable Segmentation (Interactive Segmentation)

- 사람이 원하는 영역 지정시 모델이 그 부분 분할 진행. 즉, 프롬프트 기반 분할 task를 위해 train됨.

프롬프트는 점, 상자, 마스크, 텍스트 등 다양하게 존재하고, 이에 대하여 유효한 분할 마스크를 반환하는 것이 목표

- 단추같은 프롬프트 내부의 또다른 프롬프트더라도 자체만 봤을 때 합리적이어야함.

- Zero-shot Transfer

모델이 추론할 때 다양한 프롬프트에 적절히 반응하도록 훈련시키고, 프롬프트 엔지니어링을 통해 분할 task 해결

3. Architectures of SAM

- 총 3가지 구성 요소

- 1) Image Encoder

- 사전 훈련된 Vision Transformer 기반

- 이미지당 한번만 실행되어 임베딩 '미리' 계산 -> 효율성 증대

- 2) Prompt Encoder

- 점, 상자, 텍스트 같은 Sparse prompt(이산형, 간결하게 지정)와 mask같은 dense prompt(공간적, 상세한 영역 정보) 임베딩

- 3) Lightweight Mask Decoder

- 이미지 임베딩과 프롬프트 임베딩을 결합하여 분할 마스크를 예측

4. Segment Anything Data Engine

- Segmentation mask data의 부족으로 등장한 제안으로 총 3단계

- 1) Assisted-Manual(수동)

- 고전적인 상호작용 분할과 유사

- SAM을 small size segmentation dataset 학습시켜 Model이 초기 마스크를 제안하고, 이후 주석자가 수정

- Instance에서의 의미론적 제약이 없어지고, 데이터가 충분히 수집되면 모델을 재학습시키면서 마스크당 평균 주석 시간이 크게 단축

- 2) Semi-Automatic(반자동)

- 수동으로 인해 검출된 마스크로 bounding box detector 학습 후, detector로 객체 후보 박스 출력. 이를 SAM에 프롬프트로 제시하여 마스크 재검출하는데, 신뢰도가 높은 마스크를 제외하고 두드러지지 않은 mask를 annotator가 라벨링하면서 자동으로 검출하기 어려운 Mask까지 확보.

- 다양성을 높이는 데에 집중

- 3) Fully Automatic(완전 자동)

- 앞서 많은 mask를 확보하고 이를 학습하면서 모델 성능이 대폭 상승하였고, 프롬프트가 모호한 경우에도 유효한 마스크를 출력할 수 있게 설계

- > 자동화 가능

5. Experiments

- 프롬프트 엔지니어링을 사용하여 다양한 분할 작업을 5가지로 나눠 평가

1) Single Point

단일 전경점을 프롬프트로 전달해서 마스크 출력. prompt 개수가 늘어날수록 성능차이 X
모호성을 반영하면서 precision에 취약할 수 있으나, 전반적으로 RITM보다 더 깔끔한 마스크 생성

2) Edge Detection

말그대로 밝기가 변화하는 객체의 Edge 검출

3) Object Proposals

이미지 속에 객체가 있을 것이라고 생각하는 영역 출력
학습 데이터 편향 없이 효과적으로 분할

4) Instance Segmentation

이미지 속 객체 영역 출력
ViTDet은 주먹거리게 이미지 출력
특정 주석 편향을 학습하지 않고 general하고 선명하게 마스크 생성

5) Text to Mask

텍스트 프롬프트를 통해 객체 분할
CLIP의 텍스트 인코더를 통해 생성된 텍스트 임베딩을 SAM에 프롬프트로 제공

- Ablation Study

수동에서, 자동으로 데이터 수집 방식이 발전함에 따라 평균 IoU가 크게 향상
훈련 이미지 수를 늘릴수록 평균 IoU가 꾸준히 상승
인코더 규모가 커질수록 평균 IoU 성능이 증가

6. Discussion

- generality를 향상시키는 점은 좋으나, 세밀한 구조를 놓치거나 잘못 생성하는 경우 발생
- 경계가 확실하지 않아짐 (위와 유사)
- 모두 우수하다고 결론을 지었으나, text to mask task는 굉장히 약한 수준
- 시맨틱 분할(semantic segmentation)과 파노픽 분할(panoptic segmentation)을 단순한 프롬프트로 구현하는 방법은 아직 명확하지 않음

[Paper Review] Generative Adversarial Nets

25기 분석

서영우

1. Generative Model

- discriminate model은 서로 다른 부류의 데이터를 분류하고, generative model은 입력 데이터와 유사한 분포를 갖는 새로운 데이터 생성
- training data의 distribution을 근사하는 특성을 가진 모델

2. Variational AutoEncoder(VAE)

- Input을 잘 압축하는 것이 최종적인 목표고, decoder에 input과 동일한 target을 주어 지도 학습을 진행하고 Decoder에 관심을 두지 않는 Autoencoder와 달리 샘플을 generate 잘 하는 것이 목표이므로 decoder가 주요 관심. 유연히 autoencoder와 구조 유사하고 무엇보다 generative model이라는 큰 차이점 존재.
- 확률적 인코더(Probabilistic Encoder)가 평균과 표준편차를 출력하여 확률 분포를 정의 (Ex. '웃음' 특징을 latent space의 한 확률 분포로 표현)
- 미분 불가능한 상황('무작위 샘플링')을 해결하기 위해 Reparameterization Trick(noise 결합)
- Variational Inference
학습이 잘 되기 위한 z sample을 만들어내는 함수를 얻어내기 위해 Target distribution을 잘 나타내는 분포를 찾아내서 generator을 생성
- 특정 확률 분포를 따른다고 가정하고 직접 계산하는 방식으로 학습 (Explicit Density model) -> Intractable problem

3. vs CNN

- discrimination 또는 classification을 위해 고차원 input을 계층적으로 학습하고, class label로 매핑하여 차원의 저주를 극복하였고, 역전파나 dropout, relu를 통해 기울기를 잘 보존에 성공한 CNN와 달리, MLE 계산으로 인해 단방향 학습의 어려움과 활성화함수의 장점을 못살리는 상황 발생

4. Adversarial Nets

- 0과 1 사이의 noise를 이용해서 Generator은 생성해내고, Discriminator가 실존 이미지와 가상 이미지 판별하는 것을 1 epoch로 지정하여 실제 이미지의 분포와 유사하게 학습

-

- 수식의 좌측 부분이 real data distribution을 다루고, 우측 부분은 fake data distribution으로, Discriminator는 식 전체를 maximize하는 반면 Generator은 D를 1로 극대화만 진행하는 걸 목표로 우측 항에만 관여하는 minmax 게임을 진행

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

- Generator의 초기 성능이 낮은 점을 고려하여 $\log(1-D(G(z)))$ 의 형태를 $\log(D(G(z)))$ 의 형태로 학습 진행 (gradient vanishing 방지)

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Sample minibatch of m examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\mathbf{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(\mathbf{x}^{(i)}) + \log(1 - D(G(\mathbf{z}^{(i)}))) \right]$$

end for

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(\mathbf{z}^{(i)})))$$

-

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

- D는 k번 업데이트하고 G는 1번 업데이트 (학습의 불안정성 방지, 후속 연구에서도 진행)

5. Discussion

- 장점 : Markov Chain이 필요없고, 역전파만 진행, 추론 단계 생략 -> 계산적 효율성 증대
- 단점

1) Mode Collapse

수집된 데이터에 따라 휘돌리는 경향 존재. 편향 발생

2) Train Instability

log 변환 등 어느정도 보안을 할 수 있지만 G와 D가 균형을 맞추면서 경쟁하는 데에는 어려움 발생 -> 진동 현상 발생(불안정)